

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a096dd8d56226841c.jsonl`  
- SHA-256: `122be8b052cb3089e9eac76f3d2cc059cb63a30c1640348d19794fea52d59464`  
- Source modified: `2026-07-07T04:12:34+00:00`  
- Imported at: `2026-07-08T16:00:26+00:00`  
- project: `wf_ddb0b3fd-b40`  
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:10:28.571Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:

- file: backend/pipeline/detectors/wasb_infer.py
- line: 766
- summary: The 'from trackers import build_tracker' import was moved from run()'s entry into _tracker_pass_and_write, converting a fail-fast import error into a fail-after-the-full-GPU-detector-pass error on every decode backend.
- failure scenario: With an incomplete/broken WASB-SBDT checkout whose src/trackers package is missing or unimportable (but detectors/dataloaders import fine), the pre-#506 code raised ImportError seconds into run(); the new code first runs the entire detector pass (up to timeout_sec=1800s of paid L4 time for an uncached clip) and only then raises in _tracker_pass_and_write. The detection cache survives, but the paid GPU pass completes before the run is marked failed.
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur,

is already guarded, or rests on a misreading.

2. user (2026-07-07T04:10:36.812Z)

```
249:-      # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
251:-      from trackers import build_tracker # type: ignore[attr-defined]
303:+      (which has no `build_tracker`). There is NO runtime collision ? this runs in a separate
306:+      from trackers import build_tracker # type: ignore[attr-defined]
309:      tracker = build_tracker(cfg)
```

3. user (2026-07-07T04:10:38.012Z)

```
763:      (which has no `build_tracker`). There is NO runtime collision ? this runs in a separate
766:      from trackers import build_tracker # type: ignore[attr-defined]
769:      tracker = build_tracker(cfg)
```

4. user (2026-07-07T04:10:47.550Z)

```
+
+@contextmanager
+def _torch_load_compat():
+    """Scope ``torch.load`` to ``weights_only=False`` while WASB builds its detector.
+
+    WASB's ``detector.py`` calls ``torch.load(model_path)`` with no ``weights_only``
+    argument; torch 2.6 flipped that default to True, which rejects the (pre-2.6,
+    owner-staged, trusted) WASB checkpoint. The kwarg is injected only when the
+    installed torch has it AND the caller didn't pass it, so torch 1.11 (no such
+    kwarg) and explicit callers are untouched."""
+    import inspect
+
+    import torch
+
+    real_load = torch.load
+    if "weights_only" not in inspect.signature(real_load).parameters:
+        yield # torch < 1.13 ? nothing to shim
+        return
+
+    def _load(*args, **kwargs):
+        kwargs.setdefault("weights_only", False)
+        return real_load(*args, **kwargs)
+
+    torch.load = _load
+    try:
+        yield
+    finally:
+        torch.load = real_load
+
+
+    # ----- #
+    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
+    # ----- #
+@ -151,6 +212,7 @@ def manifest_compatible(
+    frames_in: int,
+    frames_total: int,
+    device: str = "cuda",
+    decode_backend: str = "cpu",
+    ) -> bool:
+    """A cache is reusable only if the run that produced it matches this run's inputs.
+
+@ -158,6 +220,11 @@ def manifest_compatible(
+    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
+    (detections can differ numerically across devices), and vice-versa.
```

```

+ ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
+ are ``cpu``-decoded, so they stay compatible with cpu runs ? but an ``nvdec_resident``
+ run never reuses a cpu cache (NVDEC decode + the GPU affine warp are trajectory-level,
+ not byte-level, equivalents), and vice-versa.
+
+ ``frames_dir`` is the **already-canonical cache key** the caller stored in the manifest
+ (``run()`` passes its ``cache_key`` here, and writes the same value into the manifest):
+ ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
+ ``video:<abspath>``
@@ -174,6 +241,7 @@ def manifest_compatible(
    and manifest.get("frames_in") == frames_in
    and manifest.get("frames_total") == frames_total
    and manifest.get("device", "cuda") == device
+    and manifest.get("decode_backend", "cpu") == decode_backend
)

@@ -487,17 +555,11 @@ def run(
    from dataloaders import build_img_transforms, build_seq_transforms
    from dataloaders.dataset_loader import ImageDataset
    from torch.utils.data import DataLoader
-
-    # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside the
-    # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore` suppresses
-    # the mypy false positive where the main env resolves `trackers` to the Roboflow package
-    # (which has no `build_tracker`). There is NO runtime collision ? this runs in a separate
-    # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow dep.
-    from trackers import build_tracker # type: ignore[attr-defined]
-    from utils import Center
-
-    from detectors import build_detector
+
+    _ensure_np_inf() # NumPy-2.x env: the tracker pass reads np.Inf
+    streaming = frame_loader is not None
+
+    cfg = _build_cfg(weights, sport, device)
@@ -550,6 +612,7 @@ def run(
    frames_in=frames_in,
    frames_total=len(frames),
    device=device,
+    decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2 stream)
)
    if resume:
        windows_done, det_by_fid = load_and_clean_cache(cache_dir)
@@ -570,6 +633,7 @@ def run(
    "frames_in": frames_in,
    "frames_total": len(frames),
    "device": device,
+    "decode_backend": "cpu",
+    "windows_total": windows_total,
+    "windows_done": windows_done,
}
@@ -598,7 +662,8 @@ def run(
    loader = DataLoader(
        ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
    )
-    detector = build_detector(cfg)
+    with _torch_load_compat(): # torch-2.6 env: WASB's torch.load needs weights_only=False
+    detector = build_detector(cfg)
+
+    from contextlib import ExitStack

@@ -682,6 +747,25 @@ def run(
    logger.info("detector cache already complete -> tracker-only re-run")

```

```

# ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
+ return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
+
+
+def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
+ """Tracker re-run over the full ordered detection cache ? trajectory CSV.
+
+ Shared verbatim by every decode backend: the tracker consumes the same per-frame
+ candidate lists regardless of how the pixels were decoded, so keeping this single
+ keeps the output contract single.
+
+ NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside the
+ `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore` suppresses
+ the mypy false positive where the main env resolves `trackers` to the Roboflow package
+ (which has no `build_tracker`). There is NO runtime collision ? this runs in a separate
+ subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow dep.
+ """
+ from trackers import build_tracker # type: ignore[attr-defined]
+
+ _ensure_np_inf() # OnlineTracker reads np.Inf (removed in NumPy 2.0)
+ tracker = build_tracker(cfg)
+ tracker.refresh()
+ rows = []
@@ -832,6 +916,320 @@ def run_video_streaming(
+ release()

+# ----- #
+# GPU-resident decode backend (#506): NVDEC decode ? affine grid_sample resize ?
+# normalize ? detector, all on the GPU (no host round-trip). Validated on a real
+# L4: ~1.7x the cv2 streaming path with rally F1 preserved (0.574 ? baseline 0.582
+# on GX010128 at the SERVED preset).
+# ----- #
+def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
+ """`F.grid_sample` sampling grid replicating `cv2.warpAffine(img, trans,
+ (inp_w, inp_h), INTER_LINEAR)` ? THE #506 lesson lives here.
+
+ The GPU resize MUST be WASB's affine warp (center/scale geometry + cv2's
+ integer-coordinates-are-pixel-centers convention), NOT a plain resize:
+ `F.interpolate`'s half-pixel mapping shifts every sample by a constant
+ sub-pixel offset, which tips the sigmoid past `score_threshold` on marginal
+ frames and silently over-detects ? F1 0.58 ? 0.07 on the hard clip GX010128,
+ while the easy clip masked it at 98% agreement. Locked by a cv2-parity unit test.
+
+ `trans_inv` is WASB's INVERSE affine (`get_affine_transform(..., inv=1)`),
+ mapping output px ? source px; each source position is normalized to [-1, 1]
+ via `gx = (2*x + 1)/W - 1` (cv2's half-pixel convention). Pure torch+numpy ?
+ no WASB imports ? so the parity test runs on any CPU box.
+
+ Returns a `[1, inp_h, inp_w, 2]` float32 grid for `F.grid_sample(...,
+ mode='bilinear', align_corners=False, padding_mode='border')`.
+ """
+ import numpy as np
+ import torch
+
+ ys, xs = torch.meshgrid(
+ torch.arange(inp_h, dtype=torch.float32),
+ torch.arange(inp_w, dtype=torch.float32),
+ indexing="ij",
+ )
+ out_coords = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3] output px
+ mt = torch.from_numpy(np.ascontiguousarray(trans_inv, dtype=np.float32)) # [2,3]
+ src = out_coords @ mt.transpose(0, 1) # [H,W,2] source px
+ gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
+ gy = (2.0 * src[..., 1] + 1.0) / orig_h - 1.0

```

```
+ return torch.stack([gx, gy], dim=-1).unsqueeze(0) # [1,H,W,2]
+
+
+def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
+    """Per-scale inverse affines exactly as WASB's ``ImageDataset`` builds
+    ``trans_outputs_inv`` (``get_transform(img, wh, inv=1)``: center = frame middle,
+    scale = max(h, w), output size halved per extra scale). Returns
```

5. user (2026-07-07T04:11:16.730Z)

```
766: from trackers import build_tracker # type: ignore[attr-defined]
```

6. user (2026-07-07T04:11:17.601Z)

```
516: def run(
750:     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
753: def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
870: def run_video_streaming(
1049: def run_video_nvdec_resident(
1230:     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
```

7. user (2026-07-07T04:11:28.473Z)

```
516     def run(
517         frames,
518         weights: str,
519         out_csv: str,
520         sport: str = "badminton",
521         limit: int = 0,
522         batch_size: int = 8,
523         num_workers: int = 4,
524         log_every_batches: int = 50,
525         cache_dir: "str | None" = None,
526         flush_every: int = 1000,
527         fresh: bool = False,
528         frame_loader=None,
529         source_key: "str | None" = None,
530         device: str = "cuda",
531         prefetch_batches: int = 0,
532     ) -> str:
533         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
534
535         Two equivalent ways to supply pixels (output is bit-identical between them):
536
537         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
538           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
           ``cv2.imread``.
539
540         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
541           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
542           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
543           reader, which returns a PIL RGB image) over the DataLoader pass so every other
544           line of
           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
           identical
545           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
           ``num_workers=0``
546           (the in-process shim + buffer can't cross worker processes).
547
548           ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
549           frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
550           after reboot still matches.
551         """
552         import time
```

```

553
554 import torch
555 from dataloaders import build_img_transforms, build_seq_transforms
556 from dataloaders.dataset_loader import ImageDataset
557 from torch.utils.data import DataLoader
558 from utils import Center
559
560 from detectors import build_detector
561
562 _ensure_np_inf() # NumPy-2.x env: the tracker pass reads np.Inf
563 streaming = frame_loader is not None
564
565 cfg = _build_cfg(weights, sport, device)
566 frames_in = int(cfg["model"]["frames_in"])
567 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
568 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
569
570 if streaming:
571     # `frames` is already the ordered list of (synthetic) frame paths.
572     if not isinstance(frames, (list, tuple)):
573         raise SystemExit(
574             "streaming mode requires `frames` to be a list of frame paths"
575         )
576     frames = list(frames)
577     frames_dir = source_key or "stream"
578 else:
579     frames_dir = frames
580     frames = sorted(
581         glob.glob(osp.join(frames_dir, "*.png"))
582         + glob.glob(osp.join(frames_dir, "*.jpg"))
583     )
584 if limit:
585     frames = frames[:limit]
586 if len(frames) < frames_in:
587     where = frames_dir if not streaming else "video stream"
588     raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
589
590 # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
591 samples = []
592 for window in build_windows(frames, frames_in):
593     annos = [
594         {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
595         for p in window
596     ]
597     samples.append({"frames": window, "annos": annos})
598 windows_total = len(samples)
599
600 # ---- cache setup / resume decision ----- #
601 if cache_dir is None:
602     cache_dir = default_cache_dir(out_csv)
603 os.makedirs(cache_dir, exist_ok=True)
604 det_jsonl, _ = _cache_paths(cache_dir)
605 cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
606 manifest = None if fresh else read_manifest(cache_dir)
607 resume = manifest_compatible(
608     manifest or {},
609     frames_dir=cache_key,
610     weights=weights,
611     sport=sport,
612     frames_in=frames_in,
613     frames_total=len(frames),
614     device=device,
615     decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2 stream)

```

8. user (2026-07-07T04:11:29.545Z)

```
1049 def run_video_nvdec_resident(
1050     video_path: str,
1051     weights: str,
1052     out_csv: str,
1053     sport: str = "badminton",
1054     limit: int = 0,
1055     batch_size: int = 8,
1056     log_every_batches: int = 50,
1057     cache_dir: "str | None" = None,
1058     flush_every: int = 1000,
1059     fresh: bool = False,
1060 ) -> str:
1061     """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
1062     warped+normalized frames straight to the detector ? decode, resize, and
1063     normalize never touch the host. CUDA-only by construction.
1064
1065     NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
1066     warp differ from cv2 at float precision) ? equivalence is trajectory-level
1067     (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
1068     rally-F1-level (the golden gate), so the detection cache is keyed by
1069     ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
1070     cache + tracker tail as ``run()`` otherwise.
1071     """
1072     import time
1073
1074     import numpy as np
1075     import torch
1076
1077     from detectors import build_detector
1078
1079     _ensure_np_inf()
1080     if not torch.cuda.is_available():
1081         raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
1082
1083     cfg = _build_cfg(weights, sport, "cuda")
1084     frames_in = int(cfg["model"]["frames_in"])
1085     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
1086     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
1087     out_scales = list(cfg["model"]["out_scales"])
1088
1089     fe = NvdecResidentFrontend(video_path, input_wh)
1090     try:
1091         total = min(fe.count, limit) if limit else fe.count
1092         if total < frames_in:
1093             raise SystemExit(f"need >= {frames_in} frames, video has {total}")
1094         windows_total = total - frames_in + 1
1095         logger.info(
1096             f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
1097             f"{windows_total} windows"
1098         )
1099
1100         # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) --- #
1101         if cache_dir is None:
1102             cache_dir = default_cache_dir(out_csv)
1103             os.makedirs(cache_dir, exist_ok=True)
1104             det_jsonl, _ = _cache_paths(cache_dir)
1105             cache_key = "video:" + osp.abspath(video_path)
1106             manifest = None if fresh else read_manifest(cache_dir)
1107             resume = manifest_compatible(
1108                 manifest or {},
1109                 frames_dir=cache_key,
1110                 weights=weights,
1111                 sport=sport,
```



```

1175         window.pop(0)
1176     if len(window) < frames_in:
1177         continue
1178     batch_fids.append([fid for fid, _ in window])
1179     batch_imgs.append(
1180         torch.cat([chw for _, chw in window], dim=0)
1181     )
1182     if len(batch_imgs) < batch_size and idx < total - 1:
1183         continue
1184     nb = len(batch_imgs)
1185     imgs = torch.stack(batch_imgs, dim=0)
1186     # .repeat (not .expand): the postprocessor calls .numpy() on
these.
1187     affine_mats = {
1188         s: m.repeat(nb, 1, 1) for s, m in trans_t.items()
1189     }
1190     batch_results, _ = detector.run_tensor(imgs, affine_mats)
1191     for ib in range(nb):
1192         frames_payload = []
1193         for ie in sorted(batch_results[ib].keys()):
1194             fid = batch_fids[ib][ie]
1195             plain = [_det_plain(d) for d in batch_results[ib][ie]]
1196             frames_payload.append([fid, plain])
1197             det_by_fid[fid].extend(plain)
1198             pending.append({"w": win_idx, "f": frames_payload})
1199             win_idx += 1
1200     done += nb
1201     n_batches += 1
1202     batch_fids, batch_imgs = [], []
1203     if len(pending) >= flush_n or done >= windows_total:
1204         _append_windows(det_f, pending)
1205         pending = []
1206         base_manifest["windows_done"] = done
1207         write_manifest(cache_dir, base_manifest)
1208     if n_batches % log_every == 0 or done >= windows_total:
1209         el = time.time() - t0
1210         new_done = done - windows_done
1211         rate = new_done / el if el > 0 else 0.0
1212         eta = (windows_total - done) / rate if rate > 0 else 0.0
1213         logger.info(
1214             f"{done}/{windows_total} "
1215             f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
1216             f"elapsed {el:.0f}s | ETA {eta:.0f}s"
1217         )
1218     if pending: # safety net; the done>=windows_total branch already
flushed
1219         _append_windows(det_f, pending)
1220         base_manifest["windows_done"] = done
1221         write_manifest(cache_dir, base_manifest)
1222     finally:
1223         det_f.close()
1224     logger.info(f"detection done in {time.time() - t0:.0f}s; running
tracker...")
1225     else:
1226         logger.info("detector cache already complete -> tracker-only re-run")
1227     finally:
1228         fe.release()
1229
1230     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
1231
1232
1233     def main():

```

```

1233 def main():
1234     ap = argparse.ArgumentParser()
1235     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1236     ap.add_argument(
1237         "--video", help="video file; frames are extracted internally (in WSL)"
1238     )
1239     ap.add_argument(
1240         "--frames_out_dir", help="where to extract frames when --video is used"
1241     )
1242     ap.add_argument(
1243         "--stream-video",
1244         action="store_true",
1245         help="FAST PATH: decode --video on the fly and feed frames straight to "
1246         "the detector (no PNG extraction). Output-equivalent to the disk path.",
1247     )
1248     ap.add_argument(
1249         "--decode-backend",
1250         default="cpu",
1251         choices=list(DECODE_BACKENDS),
1252         help="cpu (default) = the historical cv2 decode paths (disk frames or "
1253         "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1254         "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
1255         "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1256         "(trajectory-level, not byte-level, equivalence).",
1257     )
1258     ap.add_argument("--weights", required=True)
1259     ap.add_argument("--out", required=True)
1260     ap.add_argument("--sport", default="badminton")
1261     ap.add_argument(
1262         "--device",
1263         default="cuda",
1264         choices=["cuda", "mps", "cpu"],
1265         help="torch device for the detector (default cuda = the WSL parity path)",
1266     )
1267     ap.add_argument(
1268         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
1269     )
1270     ap.add_argument("--batch-size", type=int, default=8)
1271     ap.add_argument("--num-workers", type=int, default=4)
1272     ap.add_argument(
1273         "--prefetch-batches",
1274         type=int,
1275         default=0,
1276         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
1277         "bounded producer thread of this queue depth (0 = off, the parity default)",
1278     )
1279     ap.add_argument(
1280         "--log-every-batches", type=int, default=50, help="progress log cadence"
1281     )
1282     ap.add_argument(
1283         "--cache-dir",
1284         default=None,
1285         help="resumable detector cache dir (default: <out>_wasbocache next to --out)",
1286     )
1287     ap.add_argument(
1288         "--flush-every",
1289         type=int,
1290         default=1000,
1291         help="fsync the detector cache every N windows (bounds reboot loss)",
1292     )
1293     ap.add_argument(
1294         "--fresh", action="store_true", help="ignore any existing cache and start over"
1295     )
1296     args = ap.parse_args()
1297

```

```

1298     if args.decode_backend == "nvdec_resident":
1299         # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
1300         # --stream-video cv2 path nor the extract-frames fallback below may run.
1301         if not args.video:
1302             raise SystemExit("--decode-backend nvdec_resident requires --video")
1303         if args.device != "cuda":
1304             raise SystemExit(
1305                 "--decode-backend nvdec_resident requires --device cuda "
1306                 "(NVDEC decode runs on the GPU)"
1307             )
1308         run_video_nvdec_resident(
1309             args.video,
1310             args.weights,
1311             args.out,
1312             args.sport,
1313             args.limit,
1314             batch_size=args.batch_size,
1315             log_every_batches=args.log_every_batches,
1316             cache_dir=args.cache_dir,
1317             flush_every=args.flush_every,
1318             fresh=args.fresh,
1319         )
1320         return
1321
1322     if args.stream_video:
1323         if not args.video:
1324             raise SystemExit("--stream-video requires --video")
1325         run_video_streaming(
1326             args.video,
1327             args.weights,
1328             args.out,
1329             args.sport,
1330             args.limit,
1331             batch_size=args.batch_size,
1332             log_every_batches=args.log_every_batches,
1333             cache_dir=args.cache_dir,
1334             flush_every=args.flush_every,
1335             fresh=args.fresh,
1336             device=args.device,
1337             prefetch_batches=args.prefetch_batches,
1338         )
1339         return
1340
1341     frames_dir = args.frames_dir
1342     if args.video:
1343         frames_dir = extract_frames(
1344             args.video, args.frames_out_dir or (args.video + "_frames")
1345         )
1346     if not frames_dir:
1347         raise SystemExit("provide --frames_dir or --video")
1348     run(
1349         frames_dir,
1350         args.weights,
1351         args.out,
1352         args.sport,

```

10. user (2026-07-07T04:11:58.356Z)

```

backend\config\models.py:55:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0
= no timeout)
backend\config\models.py:85:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0
= no timeout)
backend\infrastructure\execution_policy.py:4:- ``run_bounded`` ? a HARD per-attempt timeout
(``op_timeout_sec``, the hang-killer) + retry-with-backoff
backend\infrastructure\execution_policy.py:42:     """An op hung past ``op_timeout_sec`` or never

```

```

became ready within ``max_wait_sec``."
backend\infrastructure\execution_policy.py:51: op_timeout_sec: float = (
backend\infrastructure\execution_policy.py:116:         if
done.wait(timeout=policy.op_timeout_sec):
backend\infrastructure\execution_policy.py:134:         f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s"
backend\infrastructure\execution_policy.py:141:         policy.op_timeout_sec,
backend\api\job_worker.py:43:# - ExecutionPolicy op_timeout_sec kills hung generate/processing
calls
backend\api\job_worker.py:44:# - detector timeout_sec kills hung WSL/GPU calls
backend\api\job_worker.py:437:         op_timeout_sec=120.0,
backend\utils\annotation_proxy.py:19:         ffmpeg_timeout_sec,
backend\utils\annotation_proxy.py:21:         ffprobe_timeout_sec,
backend\utils\annotation_proxy.py:38:         timeout = ffprobe_timeout_sec()
backend\utils\annotation_proxy.py:49:         timeout_label(ffprobe_timeout_sec()),
backend\utils\annotation_proxy.py:97:         timeout = ffmpeg_timeout_sec()
backend\utils\annotation_proxy.py:102:         timeout_label(ffmpeg_timeout_sec()),
backend\providers\gemini.py:211:         # 3. Call Gemini under a HARD per-attempt timeout
(op_timeout_sec ? the hang-killer that was
backend\providers\gemini.py:219:         op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0
backend\pipeline\audio\hit_detector.py:24:         ffmpeg_timeout_sec,
backend\pipeline\audio\hit_detector.py:57:         timeout = ffmpeg_timeout_sec()
backend\pipeline\detectors\base.py:105:         and ``timeout_sec`` attributes (TrackNetConfig and
WasbConfig both qualify).
backend\pipeline\detectors\base.py:106:         ``timeout_sec == 0`` maps to None = wait forever
(configs default 1800s).
backend\pipeline\detectors\base.py:116:         argv, capture_output=True, text=True,
timeout=(self.cfg.timeout_sec or None)
backend\utils\video.py:7:         ffmpeg_timeout_sec,
backend\utils\video.py:9:         ffprobe_timeout_sec,
backend\utils\video.py:20:         timeout = ffprobe_timeout_sec()
backend\utils\video.py:43:         timeout_label(ffprobe_timeout_sec()),
backend\utils\video.py:87:         timeout = ffmpeg_timeout_sec()
backend\utils\video.py:101:         timeout_label(ffmpeg_timeout_sec()),
backend\pipeline\segmenters\gemini.py:17:         ffmpeg_timeout_sec,
backend\pipeline\segmenters\gemini.py:172:         timeout = ffmpeg_timeout_sec()
backend\pipeline\segmenters\gemini.py:188:
timeout_label(ffmpeg_timeout_sec()),
backend\pipeline\detectors\native_wasb_runner.py:85:         timeout_sec: int = 1800 # 30 min; a
hung GPU call is killed (0 = no timeout)
backend\pipeline\detectors\native_wasb_runner.py:121:         timeout_sec=w.timeout_sec,
backend\pipeline\detectors\native_wasb_runner.py:149:         timeout=(self.cfg.timeout_sec
or None),
backend\pipeline\detectors\native_wasb_runner.py:344:         "WASB native inference
timed out after %ss.", self.cfg.timeout_sec
backend\pipeline\detectors\person_detector.py:124:         from trackers import ByteTrackTracker
backend\pipeline\stitcher\compiler.py:10:         ffmpeg_timeout_sec,
backend\pipeline\stitcher\compiler.py:12:         ffprobe_timeout_sec,
backend\pipeline\stitcher\compiler.py:133:         timeout = ffprobe_timeout_sec()
backend\pipeline\stitcher\compiler.py:154:         timeout_label(ffprobe_timeout_sec()),
backend\pipeline\stitcher\compiler.py:329:         timeout = ffmpeg_timeout_sec()
backend\pipeline\stitcher\compiler.py:516:         timeout = ffmpeg_timeout_sec()
backend\pipeline\segmenters\trajectory_hybrid.py:303:         # timeout_sec, or the detector
env yielded nothing). This is a DETECTOR failure ? NOT
backend\pipeline\detectors\tracknet_runner.py:47:         timeout_sec: int = 1800 # 30 min; kills a
hung call (0 = no timeout)
backend\pipeline\detectors\tracknet_runner.py:65:         timeout_sec=tn.timeout_sec,
backend\pipeline\detectors\tracknet_runner.py:206:         "TrackNet inference timed out
after %ss.", self.cfg.timeout_sec
backend\pipeline\detectors\wasb_infer.py:766:         from trackers import build_tracker # type:
ignore[attr-defined]
backend\pipeline\validators\resolution_fps.py:7:from backend.utils.ffmpeg import ffprobe_bin,
ffprobe_timeout_sec, timeout_label
backend\pipeline\validators\resolution_fps.py:15:         timeout = ffprobe_timeout_sec()

```

```

backend\pipeline\validators\resolution_fps.py:38:
timeout_label(ffprobe_timeout_sec()),
backend\pipeline\validators\format.py:9:      ffprobe_timeout_sec,
backend\pipeline\validators\format.py:47:      timeout = ffprobe_timeout_sec()
backend\pipeline\validators\format.py:100:      message=f"ffprobe timed out after
{timeout_label(ffprobe_timeout_sec())}.".
backend\pipeline\detectors\wasb_runner.py:50:      timeout_sec: int = 1800 # 30 min; a hung
GPU/conda call is killed (0 = no timeout)
backend\pipeline\detectors\wasb_runner.py:74:      timeout_sec=w.timeout_sec,
backend\pipeline\detectors\wasb_runner.py:191:      logger.error("WASB inference timed out
after %ss.", self.cfg.timeout_sec)
backend\pipeline\validators\pts_continuity.py:7:      ffprobe_timeout_sec,
backend\pipeline\validators\pts_continuity.py:39:      timeout = ffprobe_timeout_sec()
backend\pipeline\validators\pts_continuity.py:132:      message=f"ffprobe PTS analysis
timed out after {timeout_label(ffprobe_timeout_sec())}.".
backend\utils\ffmpeg.py:42:def _timeout_seconds(env_var: str, default: float) ->
Optional[float]:
backend\utils\ffmpeg.py:57:def ffprobe_timeout_sec() -> Optional[float]:
backend\utils\ffmpeg.py:59:      return _timeout_seconds("RALLY_FFPROBE_TIMEOUT_SEC",
DEFAULT_FFPROBE_TIMEOUT_SEC)
backend\utils\ffmpeg.py:62:def ffmpeg_timeout_sec() -> Optional[float]:
backend\utils\ffmpeg.py:64:      return _timeout_seconds("RALLY_FFMPEG_TIMEOUT_SEC",
DEFAULT_FFMPEG_TIMEOUT_SEC)

```

11. user (2026-07-07T04:12:34.669Z)

Structured output provided successfully